6. Manual del log de tramas ADS

6.1. Qué es este log

El simulador ADS incluye un **registro de comunicación** que muestra, para cada mensaje que recibe y envía:

1. La **trama completa en hexadecimal** (los bytes tal cual viajan por la red).
2. La **explicación campo a campo**: qué significa cada byte, tanto de la cabecera como del cuerpo de datos.

Esto permite aprender y verificar el protocolo ADS de Beckhoff sin necesidad de un PLC real: cada vez que un cliente (Python o C#) habla con el simulador, el log traduce ese diálogo a lenguaje humano.

6.2. Estructura de una trama ADS

Cada mensaje ADS que viaja por TCP tiene **tres partes**:

```
+------------------+--------------------+------------------+
| AMS/TCP (6 bytes)| Cabecera AMS (32 B) | Datos (0..n B)   |
+------------------+--------------------+------------------+
```

Cabecera AMS/TCP (6 bytes)

Offset	Tamaño	Campo	Significado
0	2	`reservado`	Siempre `0x0000`.
2	4	`longitud`	Tamaño total de lo que sigue: cabecera AMS (32) + datos. Little-endian.

Cabecera AMS (32 bytes)

Offset	Tamaño	Campo	Significado
0	6	`target_netid`	AmsNetId del destino (ej. `127.0.0.1.1.1`).
6	2	`target_port`	Puerto AMS del destino (ej. `851` = PLC runtime).
8	6	`source_netid`	AmsNetId del origen.
14	2	`source_port`	Puerto AMS del

			origen.
16	2	`command_id`	Comando ADS (ver sección 6.4).
18	2	`state_flags`	`0` = solicitud, `1` = respuesta, `4` = notificación.
20	4	`length`	Bytes de datos que siguen a la cabecera.
24	4	`error_code`	Código de error (ver sección 6.6).
28	4	`invoke_id`	Identificador de transacción (la respuesta repite el mismo).

Todos los números van en **little-endian**: por ejemplo, el puerto `851` se

codifica como los bytes `53 03` (0x0353).

6.3. Cómo se lee el log

El log separa cada mensaje en bloques:

- **`== RECIBIDO · N bytes ==`** → trama que llega desde un cliente.
- **`== ENVIADO · N bytes ==`** → trama de respuesta del simulador.
- **`HEX:`** → la trama completa en hexadecimal (16 bytes por línea).
- **`[AMS/TCP]`** → los 6 primeros bytes (reservado + longitud).
- **`[CABECERA AMS]`** → los 32 bytes siguientes, campo a campo.
- **`[DATOS]`** → el cuerpo, interpretado según el comando (nombre de variable,

bytes a leer, estado, etc.).

6.4. Comandos (command_id)

Valor	Nombre	Qué hace
`0x0001`	READ_DEVICE_INFO	Pide la versión y el nombre del dispositivo.
`0x0002`	READ	Lee datos (por nombre, por handle o por índice/offset).
`0x0003`	WRITE	Escribe datos.
`0x0004`	READ_STATE	Lee el estado del dispositivo.
`0x0005`	WRITE_CONTROL	Cambia el estado (RUN/STOP…).
`0x0006`	ADD_DEVICE_NOTIFICATION	Suscribe una notificación.

0x0007	DEL_DEVICE_NOTIFICATION	Cancela una notificación.
0x0008	DEVICE_NOTIFICATION	Entrega de una notificación.
0x0009	READ_WRITE	Lee y escribe en una sola transacción.

6.5. Grupos de índice (index_group)

Los comandos READ/WRITE llevan al inicio de sus datos un `index_group` que indica cómo interpretar la petición:

Valor	Nombre	Significado
0xF000	SYMTAB	Tabla de símbolos.
0xF003	SYM_HNDBYNAME	Obtener handle a partir del nombre.
0xF004	SYM_VALBYHND	Leer/escribir valor por handle.
0xF005	SYM_VALBYNAME	Leer/escribir valor por nombre.
0xF006	SYM_INFOBYNAME	Información de un símbolo por nombre.
0xF014	SYM_RELEASEHND	Liberar un handle.
0xF020	IOIMAGE_RWIB	Imagen de proceso: entradas.
0xF030	IOIMAGE_RWOB	Imagen de proceso: salidas.
0xF080	SUMUP_READ	Lectura múltiple.
0xF081	SUMUP_WRITE	Escritura múltiple.

6.6. Estados y códigos de error

Estados del dispositivo (`ads_state`):

Valor	Nombre
0	INVALID
1	IDLE
2	RESET
5	RUN (en marcha)
6	STOP (parado)
11	ERROR

Códigos de error habituales (`error_code`):

Valor	Significado
0x00000000	Sin error

0x00000008	Comando desconocido
0x00000701	Servicio no soportado
0x00000702	Grupo de índice inválido
0x00000703	Offset inválido
0x00000705	Tamaño de parámetro incorrecto
0x00000710	Símbolo no encontrado
0x00000714	Handle de notificación inválido

6.7. Ejemplos reales comentados

Ejemplo 1 — Leer el estado (READ_STATE)

Petición (38 bytes):

```
== RECIBIDO · 38 bytes ==
  HEX:
    00 00 20 00 00 00 7f 00 00 01 01 01 53 03 7f 00
    00 01 01 01 1a a7 04 00 00 00 00 00 00 00 00 00
    00 00 01 00 00 00
  [AMS/TCP] reservado=0x0000 | longitud=32 bytes (32 cabecera AMS + 0 datos)
  [CABECERA AMS]
    target_netid = 127.0.0.1.1.1
    target_port  = 851 (PLC Runtime 1 (TC3))
    source_netid = 127.0.0.1.1.1
    source_port  = 42778
    command_id   = 0x0004 -> READ_STATE (leer estado)
    state_flags  = 0 (solicitud)
    length       = 0 bytes de datos
    error_code   = 0x00000000 (sin error)
    invoke_id    = 1
```

Lectura byte a byte:

Bytes	Interpretación
00 00	Reservado AMS/TCP = 0.
20 00 00 00	Longitud = 0x20 = 32 (cabecera AMS, sin datos).
7f 00 00 01 01 01	target_netid = 127.0.0.1.1.1.
53 03	target_port = 0x0353 = 851 (PLC runtime).
7f 00 00 01 01 01	source_netid = 127.0.0.1.1.1.
1a a7	source_port = 0xA71A = 42778.
04 00	command_id = 0x0004 (READ_STATE).
00 00	state_flags = 0 (solicitud).
00 00 00 00	length = 0 (sin datos).
00 00 00 00	error_code = 0.
01 00 00 00	invoke_id = 1.

Respuesta (42 bytes):

```
== ENVIADO · 42 bytes ==
  HEX:
    00 00 24 00 00 00 7f 00 00 01 01 01 1a a7 7f 00
    00 01 01 01 53 03 04 00 01 00 04 00 00 00 00 00
    00 00 01 00 00 00 05 00 00 00
  [AMS/TCP] reservado=0x0000 | longitud=36 bytes (32 cabecera AMS + 4 datos)
  [CABECERA AMS]
    target_netid = 127.0.0.1.1.1
    target_port  = 42778 (puerto de aplicación)
    source_netid = 127.0.0.1.1.1
    source_port  = 851
    command_id   = 0x0004 -> READ_STATE (leer estado)
    state_flags  = 1 (respuesta)
    length       = 4 bytes de datos
    error_code   = 0x00000000 (sin error)
    invoke_id    = 1
  [DATOS] ads_state=5 (RUN (en marcha)) | device_state=0
```

Observa que en la respuesta el servidor **intercambia** destino y origen

(`target` ahora es el cliente, `source` es el PLC), mantiene el mismo

`invoke_id` (1), pone `state_flags = 1` (respuesta) y devuelve 4 bytes de datos:

`05 00 00 00` → ads_state = 5 (RUN) y device_state = 0.

Ejemplo 2 — Leer una variable por nombre (MAIN.nCounter)

```
== RECIBIDO · 67 bytes ==
  ...
  [CABECERA AMS]
    command_id   = 0x0002 -> READ (leer datos)
    state_flags  = 0 (solicitud)
    ...
  [DATOS] index_group=0xf005 (SYM_VALBYNAME (valor por nombre)) | index_offset=0
    nombre="MAIN.nCounter" (longitud=13)
    bytes_a_leer=2
```

Interpretación del `[DATOS]`:

Campo	Significado
`index_group=0xf005`	`SYM_VALBYNAME`: acceso por nombre.
`index_offset=0`	Sin offset (por nombre no se usa).
`nombre="MAIN.nCounter"`	Nombre de la variable (13 bytes).
`bytes_a_leer=2`	La variable es `UINT` (2 bytes).

La respuesta trae esos 2 bytes como valor crudo:

```
== ENVIADO · 40 bytes ==
  ...
  command_id   = 0x0002 -> READ (leer datos)
  state_flags  = 1 (respuesta)
  ...
  [DATOS] respuesta: 2 bytes (valor/datos crudos)
```

Ejemplo 3 — Escribir una variable por nombre (MAIN.bStart)

```
== RECIBIDO · 66 bytes ==
  ...
  command_id   = 0x0003 -> WRITE (escribir datos)
  state_flags  = 0 (solicitud)
  ...
  [DATOS] index_group=0xf005 (SYM_VALBYNAME (valor por nombre)) | index_offset=0
    nombre="MAIN.bStart" (longitud=11)
    longitud_valor=1
```

Aquí el `[DATOS]` indica que se escribe la variable `MAIN.bStart` (11 bytes de nombre) con 1 byte de valor (un `BOOL`). La respuesta es una confirmación sin datos:

```
== ENVIADO · 38 bytes ==
  ...
  command_id   = 0x0003 -> WRITE (escribir datos)
  state_flags  = 1 (respuesta)
  length       = 0 bytes de datos
```

6.8. Cómo verlo en marcha

3. Arranca el simulador (interfaz Flutter) y pulsa **Iniciar** con puerto `40000`.
4. Lanza un cliente:

```
cd C:\DeepSeek Harness\libreriaAds\LibPhithon
python examples\verify_simulator.py 40000
```

5. Observa el panel de **log** del simulador: cada petición del cliente aparece como `RECIBIDO` y cada respuesta como `ENVIADO`, con su hex y su explicación.

6.9. Resumen rápido de lectura

En el log ves…	**Significa…**
`reservado`	Byte de alineación, siempre 0.
`longitud`	Tamaño del mensaje (cabecera + datos).
`target` / `source`	Dirección AMS de destino/origen (NetId + puerto).
`command_id`	La operación solicitada.
`state_flags = 0/1/4`	Solicitud / respuesta / notificación.
`error_code`	0 si todo fue bien; si no, el motivo.
`invoke_id`	Empareja cada petición con su respuesta.